



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo “ESCOM”

Valencia Villanueva Adrian

2026630144

Estado del Arte

Artículos Científicos

1.- TiDB: A Raft-based HTAP Database

Cita: Huang, D., Liu, Q., Cui, Q., Fang, Z., Ma, X., Xu, F., Shen, L., Tang, L., Zhou, Y., Huang, M., Wei, W., Liu, C., Zhang, J., Li, J., Wu, X., Song, L., Sun, R., Yu, S., Zhao, L., Cameron, N., Pei, L., & Tang, X. (2020). TiDB: A Raft-based HTAP database. Proceedings of the VLDB Endowment, 13(12), 3072–3084. <https://doi.org/10.14778/3415478.3415535>

●**Problema central:** Las cargas de trabajo transaccionales (OLTP) y analíticas (OLAP) tradicionalmente compiten por los mismos recursos cuando conviven en un solo motor, o requieren pipelines de ETL hacia un almacén separado que introducen retraso (staleness) en los datos analíticos.

●**Metodología:** Una arquitectura de almacenamiento multi-Raft es un almacén orientado a fila (TiKV) replicado mediante consenso Raft para servir OLTP, junto con réplicas learner asíncronas que transforman esos mismos datos a formato columnar (TiFlash) para servir OLAP fresco y consistente, una única capa SQL decide en tiempo de consulta qué formato de réplica leer.

●**Hallazgos:** Bajo el benchmark híbrido CH-benCHmark, TiDB logra desempeño aislado y alto tanto en OLTP como en OLAP de forma simultánea, sin que una carga degrade a la otra.

●**Conexión con los fundamentos:** Es una implementación práctica de la separación de niveles físico/lógico de ANSI-SPARC llevada al extremo de mantener dos representaciones físicas distintas del mismo esquema conceptual, sincronizadas por consenso distribuido.

●**Relevancia para sistemas inteligentes:** Un feature store de un sistema de ML frecuentemente necesita que una escritura transaccional (una nueva interacción de usuario) esté disponible casi de inmediato para una agregación analítica (una feature recalculada), este patrón HTAP es exactamente ese caso de uso.

2.- CockroachDB: The Resilient Geo-Distributed SQL Database

Cita completa (APA 7.^a ed.): Taft, R., Sharif, I., Matei, A., VanBenschoten, N., Lewis, J., Grieger, T., Niemi, K., Woods, A., Birzin, A., Poss, R., Bardea, P., Ranade, A., Darnell, B., Gruneir, B., Jaffray, J., Zhang, L., & Mattis, P. (2020). CockroachDB: The resilient geo-distributed SQL database. En Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (pp. 1493–1509). ACM. <https://doi.org/10.1145/3318464.3386134>

●**Problema central:** Organizaciones con usuarios en múltiples continentes necesitan bases de datos OLTP escalables que respeten restricciones regulatorias de residencia de datos (ej. GDPR) sin sacrificar alta disponibilidad ni consistencia fuerte, y sin depender de hardware especializado (como relojes atómicos, a diferencia de Google Spanner).

●**Metodología:** Un modelo transaccional novedoso sobre un almacén clave-valor replicado por rangos usando consenso Raft, con soporte de aislamiento serializable y control fino de ubicación geográfica de los datos, compatibilidad con el protocolo de cable de PostgreSQL.

●**Hallazgos:** Evaluaciones de desempeño escalan hasta 100,000 almacenes (warehouses) del benchmark TPC-C con pérdida de eficiencia mínima frente a alternativas comerciales, los autores documentan lecciones aprendidas sobre el ajuste de Raft y la decisión de priorizar aislamiento serializable sobre otros niveles más débiles.

●**Conexión con los fundamentos:** extiende la teoría clásica de control de concurrencia y atomicidad (sección 4.2) a un entorno geo-distribuido, donde el "archivo plano" que se busca evitar ya no es un archivo local sino una réplica desincronizada entre continentes.

●**Relevancia para sistemas inteligentes:** Infraestructura de entrenamiento e inferencia multi-región requiere que metadatos de gobernanza de modelos (versiones, permisos, linaje de datos) permanezcan consistentes globalmente, sin importar desde qué región se consulten.

3.- Milvus: A Purpose-Built Vector Data Management System

Cita completa (APA 7.^a ed.): Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Guo, X., Li, C., Xu, X., et al. (2021). Milvus: A purpose-built vector data management system. En Proceedings of the 2021 International Conference on Management of Data (pp. 2614–2627). ACM. <https://doi.org/10.1145/3448016.3457550>

●**Problema central:** Las librerías especializadas de búsqueda de vecino más cercano aproximado (como FAISS) resuelven el problema algorítmico de la similitud vectorial, pero no ofrecen las garantías de un sistema de gestión de datos completo: actualizaciones dinámicas, escalabilidad horizontal, ni consultas que combinen similitud vectorial con filtros sobre atributos estructurados.

●**Metodología:** Proponen un sistema de gestión de datos vectoriales de propósito específico, con SDKs y API REST, ejecución híbrida CPU/GPU, actualizaciones escalables sobre datos vectoriales dinámicos a gran escala, y soporte de consultas avanzadas más allá de la búsqueda pura por similitud.

●**Hallazgos:** Los benchmarks reportados muestran mejoras de desempeño de hasta 100x frente a sistemas comparables como Vearch y SPTAG.

●**Conexión con los fundamentos:** Es la evolución directa, como sistema de gestión de datos formal, de la categoría "especializada vectorial", aplica principios clásicos de diseño de DBMS (motor de almacenamiento, optimizador, indexación) a un tipo de dato “el vector de embedding” ausente en la teoría relacional original.

●**Relevancia para sistemas inteligentes:** Es infraestructura directa para RAG y búsqueda semántica, sin un motor capaz de indexar y consultar millones de embeddings con baja latencia, la recuperación aumentada de contexto para modelos de lenguaje no es viable a escala de producción.

4.- Comparativa

Convergencia: Los tres sistemas abandonan la arquitectura monolítica de un solo nodo en favor de diseños distribuidos y replicados que desacoplan explícitamente el motor de almacenamiento de la capa de consulta/ejecución, un eco directo de la separación de responsabilidades que propone ANSI-SPARC, aplicada ahora a nivel de infraestructura distribuida en lugar de a nivel de esquema.

Divergencias técnicas: TiDB y CockroachDB persiguen garantías transaccionales estrictas (ACID, aislamiento serializable) como requisito no negociable, mientras que Milvus relaja deliberadamente la consistencia estricta a cambio de mejor recall y latencia en la búsqueda aproximada, un punto de diseño análogo al trade-off de CAP, pero aplicado a la precisión de la búsqueda por similitud en vez de a conflictos de escritura concurrente.

Problema abierto: Ninguno de los tres resuelve por sí solo la unificación de cargas transaccionales, analíticas y de búsqueda vectorial bajo un único modelo de consistencia y un único lenguaje de consulta. Un motor verdaderamente unificado capaz de unir (join) datos relacionales estructurados con búsqueda por similitud de embeddings dentro de una misma frontera transaccional sigue siendo una dirección de investigación abierta, particularmente relevante para el patrón de RAG sobre datos operacionales que las aplicaciones nativas de IA demandan cada vez con más frecuencia.